

Assistive Technologies for Teaching Programming to Visually Impaired Learners: A Systematic Review

Felipe Honorato Mendes*, Giordana de Farias F. B. Bucci*, Ana Luísa de Bastos Chagas*,
Juliana Felix*[†], Fabrizzio Soares*

*Instituto de Informática, Universidade Federal de Goiás, Brazil

[†]Escola Politécnica e de Artes, Pontifícia Universidade Católica de Goiás, Brazil

{felipe.honorato, giordanabucci, analuisa23}@discente.ufg.br

{julianafelix, fabrizzio}@ufg.br

Abstract—Assistive technologies enable individuals with special needs to access resources that would otherwise be inaccessible due to their condition. In this context, people who are blind or visually impaired face daily challenges related to mobility, communication, and, most notably, access to education in specialized fields. In this work, we perform a systematic literature review that aims to identify and map assistive technologies designed to support programming education for individuals with visual impairments. The review was conducted across three academic databases and resulted in the selection of 22 articles that address the research questions posed in this study.

Index Terms—Assistive technology, blind, visual impaired, programming, education

I. INTRODUCTION

In recent years, a growing number of blind and visually impaired individuals have been engaging in areas previously deemed inaccessible, such as science, engineering, and technology-driven fields. The integration of 3D printing in astronomy [1], tactile neurointerfaces [2], and customized science-learning tools [3] illustrate how previously visual-only domains are becoming inclusive.

Assistive technologies refer to systems and devices designed to enhance the functional capabilities of individuals with disabilities. For the blind and visually impaired community, this includes tools such as screen readers like NVDA and JAWS, Braille displays, smart canes, and mobile navigation apps that provide real-time auditory cues. These tools have revolutionized access to reading, communication, and independent mobility [4], [5]. However, programming, as a field rooted in logic, abstract reasoning, and precise syntax, remains a significant frontier where accessibility challenges persist.

Programming presents a unique set of challenges for visually impaired learners, largely due to its highly abstract nature and dependence on visual elements. Unlike reading or navigation, coding involves interacting with invisible data structures, logical branching, and syntax that is typically understood and interpreted visually, such as color-coded keywords, indentation, and braces. These visual cues convey structural information that screen readers often fail to translate adequately, leading to confusion and difficulties in navigating code [6]. Inaccessible documentation, visually dense interfaces, and feedback mechanisms that rely heavily on visual

cues further exacerbate the exclusion of blind students in programming courses.

To foster equitable access in programming education, it is crucial to design pedagogical tools and teaching strategies that consider the specific needs of individuals with visual impairments in the context of learning to program. This paper presents a Systematic Review of the Literature (SRL) that identifies and maps existing assistive technology solutions that are already being applied to support the teaching and learning of programming specifically for people with visual impairments. By cataloging these programming-focused innovations and analyzing their strengths and limitations, the study seeks to provide a solid foundation for the development of future educational tools and inclusive policies in computer science education.

The remaining sections of this paper are structured as follows. The protocol for the Systematic Review of the Literature is presented in Section II. The research analysis is presented in Section III along with the answers obtained for the defined research questions. Section IV concludes the paper, outlining potential directions and opportunities for future research.

II. SYSTEMATIC REVIEW OF THE LITERATURE

A Systematic Review of the Literature is a form of secondary research that aims to identify, evaluate and interpret available studies that are relevant to a particular research question, area or phenomenon of interest. According to Kitchenham's [7] guidelines, this review was conducted in three main phases: planning, execution and research analysis.

A. Planning

The planning phase began with an exploratory study in well-established academic databases such as IEEE Xplore and Scopus to identify relevant articles to support the development of this review.

1) Research questions:

- **Q01.** What are the existing assistive technologies to aid in teaching programming?
- **Q02.** What types of technologies are used to support programming teaching?
- **Q03.** To which groups of people are these technologies being applied or explored?

- **Q04.** In which programming areas are these solutions/technologies being applied?

2) *Databases:* this study was conducted using three consolidated academic databases in the computer science scope, which are ACM Digital Library¹, IEEE Xplore Digital Library² and Scopus³.

3) *Inclusion criteria:* studies that presented new technologies for teaching programming to the blind or visually impaired students was the unique criteria.

4) *Exclusion criteria:*

- E1. The paper is not available;
- E2. The paper does not present any assistive technology for visually impaired people to aid programming;
- E3. The paper was not written in English;
- E4. The paper does not answer any of our research questions; and
- E5. The paper is a systematic or technology review.

5) *Quality criteria:* only studies that developed new technologies, tools or methods with the aim of enabling visual impaired people to participate in programming courses and activities.

B. Execution

An initial scoping study was conducted to refine the protocol, focusing on keywords for the search string and the inclusion and exclusion criteria. Through this preliminary search, we identified relevant studies in the field of assistive technologies for visually impaired individuals, which helped in defining the search string using commonly used terms in the literature and key concepts related to the research questions. As a result, the following search string was established and used across the databases:

```
(assistive OR accessibility) AND (technolog*
OR method* OR tool*) AND ("visual
disabilit*" OR blind* OR "low vision" OR
"visual impair*") AND (programming OR coding
OR "software development" OR algorithm
OR "Computer Science") AND (education OR
learning OR teaching)
```

This search string was applied to article titles, abstracts, and keywords (also known as ‘TAK’) in the ACM Digital Library and Scopus databases, and to the full text and metadata in the IEEE Xplore database.

The survey was conducted on May 14, 2025, yielding a total of 708 papers from the initial search. After applying the inclusion and exclusion criteria, and reviewing the titles and abstracts, 39 papers were left for full-text reading. Of these, 22 remained for data extraction and analysis for this review. Among the excluded articles, 84 duplicates were removed, 596 were excluded based on criteria E2, E4, and E5, 3 were excluded based on criterion E1, and the remaining 3 papers were rejected due to criterion E3. This selection process is summarized in Figure 1.

¹<https://dl.acm.org/>

²<https://ieeexplore.ieee.org>

³<https://www.scopus.com/>

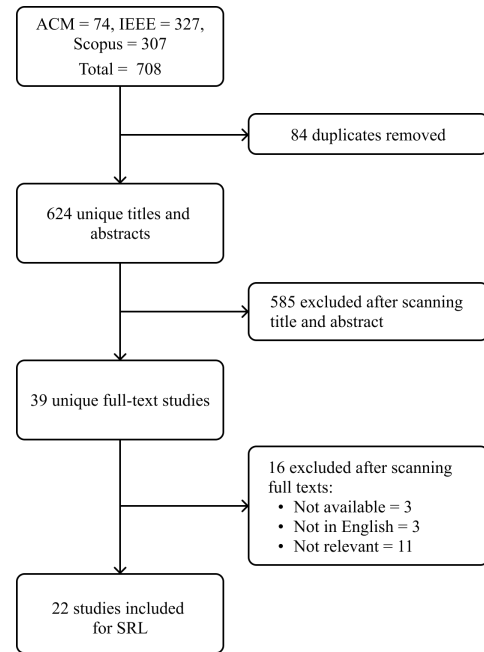


Fig. 1. Chart of the applied protocol with the number of selected papers in each phase.

III. RESEARCH ANALYSIS

The selected papers are well distributed across the years, ranging from 2002 to 2025, as illustrated in Figure 2. And, after thoroughly reviewing the 22 selected papers, we were able to answer the formulated research questions described in Section II-A.

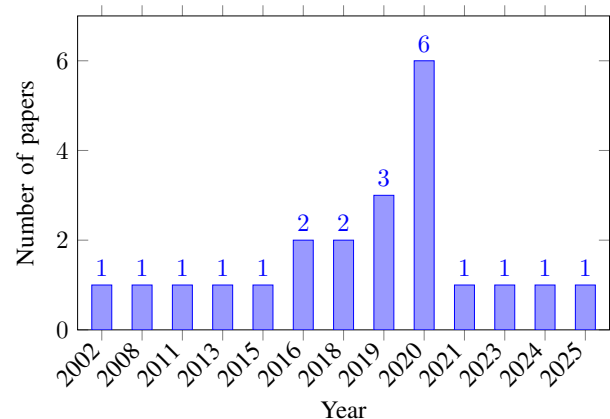


Fig. 2. Count of papers by publication year

A. **Q01)** What are the existing assistive technologies to aid in teaching programming?

Among the assistive technologies proposed for teaching programming to individuals with visual impairments, most rely primarily on tactile models. The study by Capovilla et

al. [8] used a tactile model constructed with toy bricks and numbered plates to facilitate algorithmic thinking. Crockett and Gannod [9] developed tactile diagrams to teach data structures such as trees, heaps, and queues.

Villar et al. [10] and India et al. [11] presented Project Torino, a physical programming environment composed of tactile blocks with audio feedback, specifically designed to foster computational thinking. The work by Utreras and Pontelli [12] proposed a tangible programming tool using music output and LEGO-like blocks with braille and 3D symbols to teach basic programming concepts.

Seo and Richard [13] conducted a three-day workshop using KIBO, a wooden robotics kit designed to teach fundamental programming concepts to visually impaired high school students. To enhance accessibility, they incorporated braille labels onto the programming blocks. The work by Barbareschi et al. [14] introduced TIP-TOY, an open-source tactile toolkit designed to teach foundational programming principles such as sequencing, events, and loops through musical interaction.

Rong et al. [15] developed CodeRhythm, a tangible programming toolkit that uses magnetic blocks to teach fundamental programming concepts through the creation of simple melodies. The system includes syllabic blocks (e.g., do, re, mi) and function blocks representing structures such as switches and loops. Koushik et al. [16] introduced StoryBlocks, a tangible programming game for creating audio stories, tested with blind students and educators. It enhanced engagement and understanding of sequencing.

Milne and Ladner [17] developed Blocks4All, an accessible block-based environment emphasizing tactile and spatial interaction, helping children understand code structures independently. Papazafirooulos et al. [18] used 3D-printed tactile arrays to teach sorting algorithms, which improved comprehension and engagement among blind and sighted students.

In addition to tactile interaction, several technologies incorporate audio feedback, combining modalities to enhance accessibility. This combination is evident in the studies by Ludi and Jordan [19], Stefik et al. [20], and Francioni and Smith [21] where programming environments include both tactile interaction and auditory responses. Furthermore, the study by Ludi and Reichlmayr [22] demonstrates the use of existing technologies—such as Lego Mindstorms NXT, the Not eXactly C (NXC) programming language, and the BricxCC development environment—to teach robot programming during an educational workshop for students with visual impairments.

Technologies based on screen readers were also common. Takamatsu et al. [23] enhanced visual interfaces for block-based programming with auditory navigation and code summarization. Do Nascimento et al. [24] introduced iVProg4All, a form-based visual programming environment tailored for screen readers. Schanzer et al. [25] presented CodeMirror-Blocks, a screen reader tool designed to support users in navigating code more effectively. Kane et al. [26] presented Bonk, an inclusive programming system enabling blind students to

develop audio games.

Instructional approaches centered around structured courses and workshops have also been used to teach programming to individuals with visual impairments. Koh et al. [27] developed accessible maker-based activities and coding curriculum around tools such as SnapCircuits, CodeJumper, and Snapino to teach computational thinking. The study by Ferrari et al. [28] conducted a seven-day web development course in Uganda using JAWS, tactile diagrams, and accessible editors. Finally, McMillan and Rodda-Tyler [29] reported on a mentorship program where university students taught blind high schoolers using accessible languages like Quorum and C.

Thus, assistive technologies for teaching programming range from physical and tangible methods to digital solutions with multisensory support, as well as course-based instructional approaches designed to provide structured learning experiences. This variety highlights the importance of adapting educational resources to the specific needs of students with visual impairments.

B. Q02) What types of technologies are used to support programming teaching?

Various types of technologies have been developed to support programming education for blind and visually impaired (BVI) learners. These can be broadly categorized into three groups: methods, systems, and devices. Methods refer to comprehensive teaching strategies or environments that combine hardware and software, often implemented through structured workshops. Systems are standalone software solutions or user interfaces specifically designed to accommodate non-visual interaction. Devices are tangible or physical tools with tactile features intended to aid code construction or conceptual understanding. A summary of the technologies identified across these categories is presented in Figure 3.

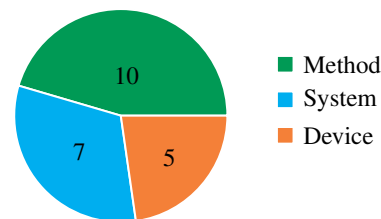


Fig. 3. Count of papers by type of technology.

Methods often involve a combination of tools, pedagogical strategies, and inclusive curricula. For example, Project Torino [11] features a physical coding system with customized blocks and connectors, tailored to support children with diverse visual abilities. These multisensory techniques not only enhance engagement but also encourage collaboration, build self-confidence, and support conceptual development through hands-on, exploratory learning. Similarly, the activities presented by Seo and Richard [13] and Koh et al. [27] illustrate how robotics workshops and maker-style learning

environments can effectively teach foundational programming concepts, such as loops and conditionals.

Systems focus on accessible user interfaces and software adaptations. CodeMirror-Blocks (CMB) [25], for instance, enhances code navigation by providing screen-reader-accessible abstract syntax tree (AST) structures, allowing users to understand and interact with code using simplified spoken descriptions. Another system, iVProg4All [24], offers a form-based visual programming environment navigable with screen readers, allowing users to create and manipulate programming elements through text fields and buttons. Similarly, an alternative block-based interface reimagines traditional visual programming environments to provide line-based navigation and code summarization features optimized for blind users, as proposed by Takamatsu et al. [23].

Devices involve tangible technologies like CodeJumper, SnapCircuits, KIBO, and LEGO-based programming kits with braille or embossed symbols. These tools enable users to construct code physically, often with magnetic connections and audio feedback, allowing them to engage with programming concepts through touch and sound. They are particularly effective for learners without prior experience, helping to scaffold learning and increase accessibility. These physical platforms have shown success in improving user engagement and understanding, particularly when supplemented by auditory and tactile feedback. Similarly, TIP-TOY [14] and CodeRhythm [15] use tangible interfaces combined with sound and touch feedback to introduce programming principles through musical interaction, further highlighting the potential of multisensory approaches in accessible computing education.

In conclusion, effectively supporting blind and visually impaired (BVI) learners in programming necessitates a comprehensive ecosystem comprising integrated methodologies, adaptive technologies, and accessible interfaces. Each component contributes uniquely to various dimensions of the learning experience, ranging from the development of conceptual knowledge to the navigation and debugging of code, collectively advancing a more inclusive and equitable educational environment.

C. Q03) To which groups of people are these technologies being applied or explored?

The assistive technologies examined were applied to a wide range of participant groups. While most interventions targeted teenagers and adults, there was also significant attention given to children. This is shown in Figures 4 as it presents the distribution of participant age groups across studies.

In addition, the majority of the studies conducted experiments with individuals who had little or no prior experience in programming, as shown in Figure 5. Some studies, however, focused on participants with advanced experience, while others adopted mixed-ability approaches that included both novice and experienced programmers.

The studies also considered the variations in the visual abilities of the participants. A large proportion of the experiments were conducted with people with mixed levels of

vision impairment, a considerable number involved only blind participants, and a smaller set focused on individuals with low vision. This distribution is presented in Figure 6, where “Not reported” indicates that the paper did not specify the type of visual impairment among the participants.

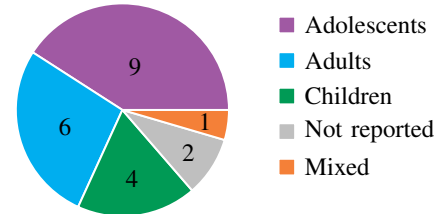


Fig. 4. Count of papers by age of participants.

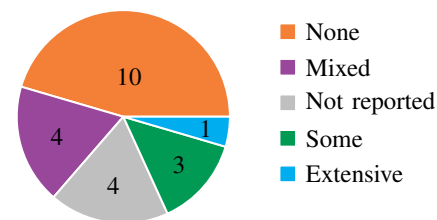


Fig. 5. Count of papers by level of programming experience.

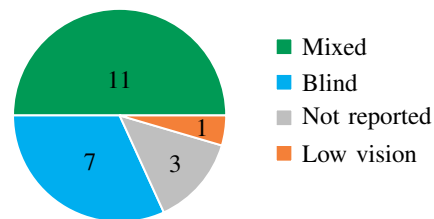


Fig. 6. Count of papers by level of visual impairment.

In the study by Utreras et al. [12], the experiment was conducted with nine adult participants, with an average age of 52.6 years, all presenting different levels of visual impairment and no prior experience with block-based programming. In contrast, Seo et al. [13] focused on adolescents aged between 16 and 19, including two blind participants, two with severe visual impairment, and one with moderate visual impairment. Furthermore, Schanzer et al. [25] chose to work with blind participants who already had some programming experience and were familiar with screen readers. In the study by Barbareschi et al. [14], the participants were children, with only one being blind, while the others were either sighted or had partial vision.

Thus, the assistive technologies explored in the reviewed studies demonstrate broad applicability across diverse groups of individuals – ranging from children to adults, including both blind and sighted participants, as well as those with or without prior programming experience –, highlighting their

strong potential for inclusion and adaptation to the varied needs of learners.

D. Q04) In which programming areas are these solutions/technologies being applied?

The solutions and technologies analyzed in the studies have been applied across a variety of programming domains, demonstrating their adaptability and flexibility to different areas within computer science education, as summarized in Figure 7. Although the predominant focus has been on foundational programming skills – particularly algorithmic thinking and introductory programming concepts –, many solutions are designed for general-purpose programming contexts and robotics. Other application areas include web development and audio game programming.

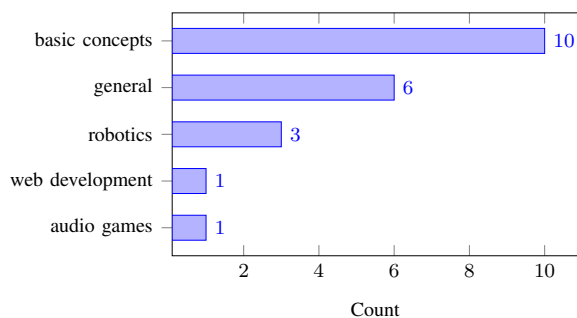


Fig. 7. Count of papers by programming area.

For example, the study by Utreras and Pontell [12] developed a tangible programming tool designed to help children learn foundational programming concepts. Do Nascimento et al. [24] introduce a general-purpose accessible programming interface tailored for blind learners. In the domain of robotics, the work by Ludi and Jordan [19] leverages LEGO Mindstorms kits to support accessible robotic programming.

Overall, these studies highlight the potential of assistive technologies to support visually impaired learners across multiple programming domains, suggesting that such tools can be effectively integrated into a wide range of educational contexts.

IV. CONCLUSIONS

This study presented a Systematic Literature Review focused on assistive technologies designed to support programming education for blind and visually impaired students. By analyzing twenty-two selected studies, we identified a range of approaches – including tactile and haptic models, accessible programming environments, and audio-based feedback systems – that aim to reduce barriers to learning programming concepts.

The diversity of approaches demonstrates that a one-size-fits-all solution is inadequate and that assistive technologies have been successfully applied across different groups of people, including individuals of varying ages, with diverse levels of visual impairment, and regardless of their prior

programming experience, in multiple learning environments. Furthermore, while some initiatives operate independently of specific programming domains, others leverage adapted environments and custom tools to enhance accessibility within traditional programming contexts.

However, despite the promising nature of tools presented, challenges remain, particularly concerning cost, scalability and the customization required to meet the diverse sensory and cognitive needs of individual learners.

A considerable portion of current assistive technologies emphasize tangible and hardware-dependent approaches, which, while effective, are often costly and difficult to scale. This highlights the urgency of developing software-driven alternatives that are more accessible, cost-effective, and distributable.

Another key issue that persists is the heavy reliance on visual elements in traditional coding interfaces, which are often poorly translated by screen readers. This limitation continues to hinder equitable access to programming education. While some tools have managed to overcome these challenges through alternative modalities such as block-based or audio-enhanced programming, robust and consolidated solutions are still lacking.

Ultimately, this review not only provides a comprehensive overview of current assistive solutions on the field but also layers the groundwork for future research and development of technologies that promote a more inclusive programming education for blind and visually impaired learners.

ACKNOWLEDGMENTS

The authors would like to acknowledge the support of FAPEG – Fundação de Amparo à Pesquisa do Estado de Goiás – Call Nº 03/2025 – Programa de Mobilidade Internacional (Participação em Eventos no Exterior); CAPES – Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Financin- g Code #001; and CAPES/PDPG n. 30/2022 – Programa Emergencial de Solidariedade Acadêmica.

REFERENCES

- [1] T. Madura, S. O’Beollain, C. Christian, T. Wild, D. Hurd, N. Grice, H. Davis, and L. Bartolone, “Astronomy 3d printed models to enhance learning for blind and visually impaired students (and everyone!),” in *Proceedings of Society for Information Technology & Teacher Education International Conference 2025*, R. H. J. Cohen, Ed. Orlando, FL, USA: Association for the Advancement of Computing in Education (AACE), March 2025, pp. 2463–2469.
- [2] F. Morelli, L. Schiatti, G. Cappagli, C. Martolini, M. Gori, and S. Signorini, “Clinical assessment of the techarm system on visually impaired and blind children during uni- and multi-sensory perception tasks,” *Frontiers in Neuroscience*, vol. 17, p. 1158438, 2023.
- [3] T. S. Herzberg and C. R. McBride, “Middle and high school students with visual impairments describe their experiences in learning a new braille code for mathematics and science,” *Journal of Visual Impairment & Blindness*, vol. 118, no. 3, pp. 141–150, 2024, original work published 2024.
- [4] E. Casanova, D. Guffanti, and L. Hidalgo, “Technological advancements in human navigation for the visually impaired: A systematic review,” *Sensors*, vol. 25, no. 7, 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/25/7/2213>
- [5] W. Seiple, H. P. A. van der Aa, F. Garcia-Piña, I. Greco, C. Roberts, and R. van Nispen, “Performance on activities of daily living and user experience when using artificial intelligence by individuals with vision impairment,” *Translational Vision Science & Technology*, vol. 14, no. 1, pp. 3–3, 01 2025.

- [6] Y. Cha, V. Jackson, K. Kohl, R. Prikładnicki, A. van der Hoek, and S. Branham, "The dilemma of building do-it-yourself (diy) solutions for workplace accessibility," 01 2025.
- [7] B. Kitchenham, "Procedures for performing systematic reviews," *Keele, UK, Keele University*, vol. 33, no. 2004, pp. 1–26, 2004.
- [8] D. Capovilla, J. Krugel, and P. Hubwieser, "Teaching algorithmic thinking using haptic models for visually impaired students," in *2013 Learning and Teaching in Computing and Engineering*. IEEE, 2013, pp. 167–171.
- [9] A. R. Crockett and G. C. Gannod, "Improving understanding of data structures for the blind with tactile media and a user-centered iterative approach," in *2020 IEEE Frontiers in Education Conference (FIE)*. IEEE, 2020, pp. 1–8.
- [10] N. Villar, C. Morrison, D. Cletheroe, T. Regan, A. Thieme, and G. Saul, "Physical programming for blind and low vision children at scale," in *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019, pp. 1–4.
- [11] G. India, G. Ramakrishna, J. Pal, and M. Swaminathan, "Conceptual learning through accessible play: Project torino and computational thinking for blind children in india," in *Proceedings of the 2020 international conference on information and communication technologies and development*, 2020, pp. 1–11.
- [12] E. Utreras and E. Pontelli, "Accessibility of block-based introductory programming languages and a tangible programming tool prototype," in *International Conference on Computers Helping People with Special Needs*. Springer, 2020, pp. 27–34.
- [13] J. Seo and G. Richard, "Coding through touch: Exploring and re-designing tactile making activities with learners with visual dis/abilities," in *Proceedings of the 14th International Conference of the Learning Sciences: The Interdisciplinarity of the Learning Sciences, ICLS 2020*. International Society of the Learning Sciences (ISLS), 2020.
- [14] G. Barbareschi, E. Costanza, and C. Holloway, "Tip-toy: a tactile, open-source computational toolkit to support learning across visual abilities," in *Proceedings of the 22nd international ACM SIGACCESS conference on computers and accessibility*, 2020, pp. 1–14.
- [15] Z. Rong, N. F. Chan, T. Chen, and K. Zhu, "Coderhythm: A tangible programming toolkit for visually impaired students," in *Proceedings of the Eighth International Workshop of Chinese CHI*, 2020, pp. 57–60.
- [16] V. Koushik, D. Guinness, and S. K. Kane, "Storyblocks: A tangible programming game to create accessible audio stories," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019, pp. 1–12.
- [17] L. R. Milne and R. E. Ladner, "Blocks4all: overcoming accessibility barriers to blocks programming for children with visual impairments," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, pp. 1–10.
- [18] N. Papazafropoulos, L. Fanucci, B. Leporini, S. Pelagatti, and R. Roncella, "Haptic models of arrays through 3d printing for computer science education," in *Computers Helping People with Special Needs: 15th International Conference, ICCHP 2016, Linz, Austria, July 13-15, 2016, Proceedings, Part I 15*. Springer, 2016, pp. 491–498.
- [19] S. Ludi and S. Jordan, "A jbrick: Accessible robotics programming for visually impaired users," in *Universal Access in Human-Computer Interaction. Access to Learning, Health and Well-Being: 9th International Conference, UAHCI 2015, Held as Part of HCI International 2015, Los Angeles, CA, USA, August 2-7, 2015, Proceedings, Part III 9*. Springer, 2015, pp. 157–168.
- [20] A. M. Stefik, C. Hundhausen, and D. Smith, "On the design of an educational infrastructure for the blind and visually impaired in computer science," in *Proceedings of the 42nd ACM technical symposium on Computer science education*, 2011, pp. 571–576.
- [21] J. M. Francioni and A. C. Smith, "Computer science accessibility for students with visual disabilities," *SIGCSE Bull.*, vol. 34, no. 1, p. 91–95, Feb. 2002. [Online]. Available: <https://doi-org.ez49.periodicos.capes.gov.br/10.1145/563517.563372>
- [22] S. A. Ludi and T. Reichlmayr, "Developing inclusive outreach activities for students with visual impairments," in *Proceedings of the 39th SIGCSE technical symposium on Computer science education*, 2008, pp. 439–443.
- [23] N. Takamatsu, E. Kamioka, C. M. Tran, and P. X. Tan, "Computer programming education system for visually impaired individuals," in *14th International Workshop on Computer Science and Engineering, WCSE 2024*. International Workshop on Computer Science and Engineering (WCSE), 2024, pp. 63–67.
- [24] M. D. Do Nascimento, A. A. Brandão, L. D. O. Brandão, and T. M. Casal, "Towards ivprog4all: an accessibility test with blind," in *2023 IEEE Frontiers in Education Conference (FIE)*. IEEE, 2023, pp. 01–05.
- [25] E. Schanzer, S. Bahram, and S. Krishnamurthi, "Accessible ast-based programming for visually-impaired programmers," in *Proceedings of the 50th ACM technical symposium on computer science education*, 2019, pp. 773–779.
- [26] S. K. Kane, V. Koushik, and A. Muehlbradt, "Bonk: accessible programming for accessible audio games," in *Proceedings of the 17th ACM conference on interaction design and children*, 2018, pp. 132–142.
- [27] K. Koh, J. Seo, S. Chen, and E. M. Cox, "Engaging with information beyond vision: hands-on approaches to computational thinking for blind and visually impaired learners," *Information Research an international electronic journal*, vol. 30, no. iConf, pp. 280–286, 2025.
- [28] C. Ferrari, A. Hurst, and S. Fitzgerald, "Blind web development training at oysters and pearls technology camp in uganda," in *Proceedings of the 16th International Web for All Conference*, 2019, pp. 1–10.
- [29] C. McMillan and A. Rodda-Tyler, "Collaborative software engineering education between college seniors and blind high school students," in *Proceedings of the 38th International Conference on Software Engineering Companion*, 2016, pp. 360–363.